

OCP Maintenance AI -- Complete Tool Listing

Auto-generated from source code inspection.
Last updated: 2026-02-22

Summary

Category	Count
Deterministic Engines	35
MCP Tool Wrappers (registered)	122
Validators	3
Processors	5
Generators	1
Total Components	166

Agent-to-tool distribution (from `AGENT_TOOL_MAP` in `server.py`):

Agent Type	Tools Assigned
orchestrator	13
reliability	48
planning	55
spare_parts	3
unassigned	4
Total mapped	123

Note: 122 unique tools are registered in `TOOL_REGISTRY` via `@tool` decorators across 26 wrapper files. Of these, 119 are assigned to agents via `AGENT_TOOL_MAP` (no tool appears in more than one agent). 4 registered tools (`get_all_entity_states` , `validate_hierarchy` , `validate_functions` , `validate_criticality_data`) are not assigned to any agent and are listed in Section B.5. One tool in the map may reference a name not yet registered as a wrapper (pending verification at runtime).

Section A -- Deterministic Engines

All engines are located under `tools/engines/` . They are pure-logic, deterministic modules that require no LLM.

#	Engine Name	File Path	Main Functions	Input
1	CriticalityEngine	tools/engines/criticality_engine.py	calculate_overall_score() , determine_risk_class() , validate_full_matrix() , assess() , assess_gfsn()	Criteria scores dict (safety environment, production, quality, frequency)
2	RCMDecisionEngine	tools/engines/rcm_decision_engine.py	decide() , _decide_hidden() , _decide_evident() , validate_frequency_unit()	RCMInput (failure mode consequence type, detectability, safety/environmental flag)
3	SAPExportEngine	tools/engines/sap_export_engine.py	generate_upload_package() , validate_cross_references() , validate_sap_field_lengths()	Work packages list, plant hierarchy nodes
4	EquipmentResolver	tools/engines/equipment_resolver.py	resolve() , _fuzzy_match_tags() , _fuzzy_match_descriptions()	Text input (voice/text/tag) equipment registry list
5	BacklogGrouper	tools/engines/backlog_grouper.py	group_by_equipment() , group_by_area() , group_by_shutdown() , find_all_groups() , stratify()	List of BacklogEntry d
6	MaterialMapper	tools/engines/material_mapper.py	suggest_materials() , validate_task_materials() , _generic_suggestion()	Component type, failure mode keyword, equipment_id, BOM regist
7	WorkInstructionGenerator	tools/engines/work_instruction_generator.py	generate() , validate_work_instruction()	Task type, component, fail mode, safety flags
8	PriorityEngine	tools/engines/priority_engine.py	calculate_priority() , validate_priority_override() , calculate_gfsn_priority()	PriorityInput (critical safety flags, failure mode, production impact, recurrence)
9	HealthScoreEngine	tools/engines/health_score_engine.py	calculate() , criticality_to_score() , backlog_pressure_score() , strategy_coverage_score() , condition_status_score() , execution_compliance_score() , determine_trend()	5-dimension input (critica backlog count, strategy coverage %, condition scc compliance %)
10	KPIEngine	tools/engines/kpi_engine.py	calculate_mtbfd() , calculate_mttr() , calculate_availability() , calculate_ooe() , calculate_schedule_compliance() , calculate_pm_compliance() , calculate_reactive_ratio() , calculate_from_records()	Operating hours, failure count, repair hours, downtime hours, performance data
11	WeibullEngine	tools/engines/weibull_engine.py	fit_parameters() , reliability() , failure_probability() , hazard_rate() , mean_life() , classify_failure_pattern() , predict()	Time-to-failure data list o shape/scale parameters
12	VarianceDetector	tools/engines/variance_detector.py	compute_stats() , z_score() , detect_variance() , detect_multi_metric() , rank_plants()	Metric values per plant, threshold z-score
13	CAPAEEngine	tools/engines/capa_engine.py	create_capa() , advance_phase() , update_status() , add_action() , set_root_cause() , is_overdue() , get_summary()	CAPA type, description, source, severity, phase transitions

#	Engine Name	File Path	Main Functions	Input
14	ManagementReviewEngine	tools/engines/management_review_engine.py	generate_review() , _compute_kpi_trends() , _generate_findings() , _generate_actions()	Plant KPI data, period, previous review data
15	RCAEngine	tools/engines/rca_engine.py	classify_event() , create_analysis() , run_5w2h() , add_cause() , classify_root_cause_level() , validate_root_cause_chain() , collect_evidence_5p() , evaluate_solution() , prioritize_solutions() , advance_status() , compute_de_kpis()	Event data, failure data, ca chains
16	PlanningKPIEngine	tools/engines/planning_kpi_engine.py	calculate()	Planning metrics dict (backlog_total, pm_completed, schedule_executed, etc.)
17	SchedulingEngine	tools/engines/scheduling_engine.py	create_weekly_program() , assign_support_tasks() , level_resources() , detect_conflicts() , validate_work_package_elements() , finalize_program() , activate_program() , complete_program() , level_resources_enhanced() , suggest_conflict_resolutions() , split_multi_day_package()	Plant ID, week/year, work packages, workforce, trad capacities
18	StateMachine	tools/engines/state_machine.py	validate_transition() , get_valid_transitions() , get_all_states()	Entity type, current state, target state
19	SparePartsEngine	tools/engines/spare_parts_engine.py	classify_ved() , classify_fsn() , classify_abc() , calculate_criticality_score() , calculate_stock_levels() , optimize_inventory()	Material data (unit_cost, annual_usage, lead_time, last_issue_date, criticality)
20	ShutdownEngine	tools/engines/shutdown_engine.py	create_shutdown() , start_shutdown() , update_progress() , complete_shutdown() , cancel_shutdown() , calculate_metrics()	Shutdown type, scope, planned duration, work packages, actual progress
21	MoCEngine	tools/engines/moc_engine.py	create_moc() , submit_moc() , start_review() , approve_moc() , reject_moc() , resubmit_moc() , start_implementation() , close_moc() , assess_risk()	Change title, description, category, impact areas
22	OCREngine	tools/engines/ocr_engine.py	calculate_optimal_interval() , sensitivity_analysis() , batch_analyze()	Failure rate, repair cost, PI cost, production loss rate, duration
23	JackKnifeEngine	tools/engines/jackknife_engine.py	analyze() , get_bad_actors()	Equipment records with MTBF and MTTR values
24	ParetoEngine	tools/engines/pareto_engine.py	analyze() , analyze_failures() , analyze_costs() , analyze_downtime()	Records with category + value fields
25	LCCEngine	tools/engines/lcc_engine.py	calculate() , compare_alternatives() ,	Capital cost, annual opera cost, maintenance cost,

#	Engine Name	File Path	Main Functions	Input
			<code>find_breakeven()</code>	discount rate, life years
26	RBIEngine	<code>tools/engines/rbi_engine.py</code>	<code>assess()</code> , <code>batch_assess()</code> , <code>prioritize_inspections()</code>	Equipment type, age, corrosion rate, last inspection date, consequence
27	DEKPIEngine	<code>tools/engines/de_kpi_engine.py</code>	<code>calculate()</code> , <code>calculate_trends()</code> , <code>assess_program_health()</code> , <code>compare_plants()</code>	Failure records, CAPA records, RCA records, period
28	NotificationEngine	<code>tools/engines/notification_engine.py</code>	<code>check_rbi_overdue()</code> , <code>check_kpi_thresholds()</code> , <code>check_equipment_risk()</code> , <code>check_backlog_aging()</code> , <code>check_overdue_actions()</code> , <code>generate_all_notifications()</code>	KPI data, RBI data, backlog data, equipment health data
29	DataImportEngine	<code>tools/engines/data_import_engine.py</code>	<code>validate_hierarchy_data()</code> , <code>validate_failure_history()</code> , <code>validate_maintenance_plan()</code> , <code>detect_column_mapping()</code> , <code>summarize_import()</code>	Raw data dicts (hierarchy rows, failure rows, plan rows)
30	DataExportEngine	<code>tools/engines/data_export_engine.py</code>	<code>prepare_equipment_export()</code> , <code>prepare_kpi_export()</code> , <code>prepare_report_export()</code> , <code>prepare_schedule_export()</code>	Equipment list, KPI data, report data, schedule data
31	CrossModuleEngine	<code>tools/engines/cross_module_engine.py</code>	<code>correlate_criticality_failures()</code> , <code>correlate_cost_reliability()</code> , <code>correlate_health_backlog()</code> , <code>find_bad_actor_overlap()</code> , <code>generate_cross_module_summary()</code>	Multi-module data (criticality failures, costs, health scores, backlog)
32	ReportingEngine	<code>tools/engines/reporting_engine.py</code>	<code>generate_weekly_report()</code> , <code>generate_monthly_kpi_report()</code> , <code>generate_quarterly_review()</code> , <code>get_report_sections()</code>	Plant ID, period, KPI data, backlog data, schedule data
33	WorkPackageAssemblyEngine	<code>tools/engines/work_package_assembly_engine.py</code>	<code>assemble_work_package()</code> , <code>check_element_readiness()</code> , <code>generate_compliance_report()</code>	Package ID, name, equipment tag, 7 element types, assembled_by
34	ExecutionTaskEngine	<code>tools/engines/execution_task_engine.py</code>	<code>build_execution_sequence()</code> , <code>get_loto_removal_checklist()</code> , <code>_topological_sort()</code> , <code>_calculate_critical_path()</code>	Package ID, support tasks with types/dependencies, package attributes (elevation, shutdown)
35	FMECAEngine	<code>tools/engines/fmeca_engine.py</code>	<code>create_worksheet()</code> , <code>add_row()</code> , <code>calculate_rpn()</code> , <code>run_stage_4_decisions()</code> , <code>advance_stage()</code> , <code>generate_summary()</code> , <code>complete_worksheet()</code>	Equipment ID/tag/name, severity/occurrence/detection (1-10), worksheet data

Section B -- MCP Tool Wrappers

All tool wrappers are located under `agents/tool_wrappers/`. Each tool is registered via the `@tool` decorator into `TOOL_REGISTRY` and exposed through `server.py`. Tools accept a single `input_json` string parameter and return a JSON string.

B.1 -- Orchestrator Agent Tools (13 tools)

#	Tool Name	Wrapper File	Engine(s) Wrapped	Description
1	<code>run_full_validation</code>	<code>validation_tools.py</code>	QualityValidator	Run complete validation across all entity types (hierarchy, functions, criticality, FMs, tasks, WPs, cross-entity).
2	<code>evaluate_confidence</code>	<code>validation_tools.py</code>	ConfidenceValidator	Evaluate AI confidence for a single entity against 4 review levels.
3	<code>batch_evaluate_confidence</code>	<code>validation_tools.py</code>	ConfidenceValidator	Batch confidence evaluation for multiple entities of the same type.
4	<code>validate_state_transition</code>	<code>state_machine_tools.py</code>	StateMachine	Check if a state transition is allowed for an entity type.
5	<code>get_valid_transitions</code>	<code>state_machine_tools.py</code>	StateMachine	Get all valid next states for an entity in its current state.
6	<code>generate_management_review</code>	<code>management_review_tools.py</code>	ManagementReviewEngine	Generate executive management review summary with KPI trends and recommendations.
7	<code>generate_weekly_report</code>	<code>reporting_tools.py</code>	ReportingEngine	Generate weekly maintenance report for a plant.
8	<code>generate_monthly_kpi_report</code>	<code>reporting_tools.py</code>	ReportingEngine	Generate monthly KPI report for a plant.
9	<code>generate_quarterly_review</code>	<code>reporting_tools.py</code>	ReportingEngine	Generate quarterly review report for a plant.
10	<code>generate_all_notifications</code>	<code>notification_tools.py</code>	NotificationEngine	Run all notification checks and return combined alerts.
11	<code>run_cross_module_analysis</code>	<code>notification_tools.py</code>	CrossModuleEngine	Run cross-module correlation analysis across maintenance data.
12	<code>suggest_conflict_resolutions</code>	<code>scheduling_tools.py</code>	SchedulingEngine	Suggest actionable resolutions for scheduling conflicts.
13	<code>get_loto_removal_checklist</code>	<code>execution_task_tools.py</code>	ExecutionTaskEngine	Get the LOTO removal safety checklist (8 items).

B.2 -- Reliability Agent Tools (48 tools)

#	Tool Name	Wrapper File	Engine(s) Wrapped	Description
1	<code>assess_criticality</code>	<code>criticality_tools.py</code>	CriticalityEngine	Assess equipment criticality with full matrix (GFSN methodology).
2	<code>calculate_criticality_score</code>	<code>criticality_tools.py</code>	CriticalityEngine	Calculate overall criticality score from criteria scores.
3	<code>determine_risk_class</code>	<code>criticality_tools.py</code>	CriticalityEngine	Determine risk class (AA/A+/A/B/C) from a numerical score.
4	<code>validate_criticality_matrix</code>	<code>criticality_tools.py</code>	CriticalityEngine	Validate a complete criticality matrix for consistency.
5	<code>rcm_decide</code>	<code>rcm_tools.py</code>	RCMDecisionEngine	Run RCM decision tree to determine maintenance strategy.
6	<code>validate_frequency_unit</code>	<code>rcm_tools.py</code>	RCMDecisionEngine	Validate that a frequency/unit combination is acceptable.
7	<code>validate_fm_combination</code>	<code>fm_lookup_tools.py</code>	VALID_FM_COMBINATIONS	Check if a mechanism+cause pair is in the 72 valid combinations.
8	<code>get_valid_fm_combinations</code>	<code>fm_lookup_tools.py</code>	VALID_FM_COMBINATIONS	Get all valid cause values for a given mechanism.
9	<code>list_all_mechanisms</code>	<code>fm_lookup_tools.py</code>	Mechanism enum	List all valid failure mechanism enum values.
10	<code>list_all_causes</code>	<code>fm_lookup_tools.py</code>	Cause enum	List all valid failure cause enum values.
11	<code>validate_failure_modes</code>	<code>validation_tools.py</code>	QualityValidator	Validate failure modes data quality (FM rules from REF-04).
12	<code>validate_tasks</code>	<code>validation_tools.py</code>	QualityValidator	Validate maintenance tasks data quality.
13	<code>validate_cross_entity</code>	<code>validation_tools.py</code>	QualityValidator	Validate cross-entity referential integrity.
14	<code>validate_fm_what</code>	<code>validation_tools.py</code>	NamingValidator	Validate failure mode "what" field naming conventions (FM-01, FM-02).
15	<code>calculate_priority</code>	<code>priority_tools.py</code>	PriorityEngine	Calculate work order priority from criticality and failure data.
16	<code>validate_priority_override</code>	<code>priority_tools.py</code>	PriorityEngine	Validate that a manual priority override is justified.
17	<code>calculate_health_score</code>	<code>health_tools.py</code>	HealthScoreEngine	Calculate composite Asset Health Index (0-100) from 5 dimensions.
18	<code>determine_health_trend</code>	<code>health_tools.py</code>	HealthScoreEngine	Determine health trend from a series of historical scores.
19	<code>fit_weibull</code>	<code>weibull_tools.py</code>	WeibullEngine	Fit Weibull distribution parameters (beta, eta) from failure data.
20	<code>predict_failure</code>	<code>weibull_tools.py</code>	WeibullEngine	Predict next failure probability using fitted Weibull parameters.
21	<code>weibull_reliability</code>	<code>weibull_tools.py</code>	WeibullEngine	Calculate reliability at a given time using Weibull parameters.
22	<code>detect_variance</code>	<code>variance_tools.py</code>	VarianceDetector	Detect outlier plants for a single KPI metric across portfolio.
23	<code>detect_multi_metric_variance</code>	<code>variance_tools.py</code>	VarianceDetector	Detect outlier plants across multiple KPI metrics simultaneously.
24	<code>rank_plants</code>	<code>variance_tools.py</code>	VarianceDetector	Rank plants by composite score across multiple KPIs.
25	<code>calculate_mtbtf</code>	<code>kpi_tools.py</code>	KPIEngine	Calculate Mean Time Between Failures.
26	<code>calculate_mttr</code>	<code>kpi_tools.py</code>	KPIEngine	Calculate Mean Time To Repair.
27	<code>calculate_availability</code>	<code>kpi_tools.py</code>	KPIEngine	Calculate equipment availability percentage.
28	<code>calculate_oeo</code>	<code>kpi_tools.py</code>	KPIEngine	Calculate Overall Equipment Effectiveness.
29	<code>calculate_kpis_from_records</code>	<code>kpi_tools.py</code>	KPIEngine	Calculate all KPIs from work order records.
30	<code>classify_rca_event</code>	<code>rca_tools.py</code>	RCAEngine	Classify an event to determine if RCA is required.
31	<code>create_rca_analysis</code>	<code>rca_tools.py</code>	RCAEngine	Create a new RCA analysis for a failure event.
32	<code>run_rca_5w2h</code>	<code>rca_tools.py</code>	RCAEngine	Run 5W2H analysis on an RCA event.
33	<code>add_rca_cause</code>	<code>rca_tools.py</code>	RCAEngine	Add a cause to the RCA causal chain.

#	Tool Name	Wrapper File	Engine(s) Wrapped	Description
34	<code>classify_root_cause</code>	<code>rca_tools.py</code>	RCAEngine	Classify root cause level (physical/human/latent/organizational).
35	<code>validate_rca_chain</code>	<code>rca_tools.py</code>	RCAEngine	Validate completeness and consistency of an RCA causal chain.
36	<code>collect_rca_evidence</code>	<code>rca_tools.py</code>	RCAEngine	Collect evidence using the 5P methodology.
37	<code>evaluate_rca_solution</code>	<code>rca_tools.py</code>	RCAEngine	Evaluate a proposed RCA solution against criteria.
38	<code>prioritize_rca_solutions</code>	<code>rca_tools.py</code>	RCAEngine	Prioritize multiple RCA solutions by feasibility and impact.
39	<code>advance_rca_status</code>	<code>rca_tools.py</code>	RCAEngine	Advance RCA analysis through its status lifecycle.
40	<code>compute_de_kpis</code>	<code>rca_tools.py</code>	RCAEngine	Compute Defect Elimination KPIs from RCA data.
41	<code>calculate_de_kpis_standalone</code>	<code>reporting_tools.py</code>	DEKPIEngine	Calculate DE KPIs independently (standalone).
42	<code>assess_de_program_health</code>	<code>reporting_tools.py</code>	DEKPIEngine	Assess overall Defect Elimination program health.
43	<code>check_rbi_overdue</code>	<code>notification_tools.py</code>	NotificationEngine	Check for overdue RBI inspections.
44	<code>check_kpi_breaches</code>	<code>notification_tools.py</code>	NotificationEngine	Check for KPI threshold breaches.
45	<code>find_bad_actor_overlap</code>	<code>notification_tools.py</code>	CrossModuleEngine	Find equipment flagged as bad actors across multiple modules.
46	<code>create_fmeca_worksheet</code>	<code>fmeca_tools.py</code>	FMECAEngine	Create a new FMECA worksheet for an equipment item.
47	<code>calculate_rpn</code>	<code>fmeca_tools.py</code>	FMECAEngine	Calculate Risk Priority Number (Severity x Occurrence x Detection).
48	<code>generate_fmeca_summary</code>	<code>fmeca_tools.py</code>	FMECAEngine	Generate FMECA summary statistics with RPN distribution and top risks.

B.3 -- Planning Agent Tools (55 tools)

#	Tool Name	Wrapper File	Engine(s) Wrapped	Description
1	group_by_equipment	backlog_tools.py	BacklogGrouper	Group backlog items by equipment tag.
2	group_by_area	backlog_tools.py	BacklogGrouper	Group backlog items by plant area code.
3	group_by_shutdown	backlog_tools.py	BacklogGrouper	Group backlog items requiring shutdown into shutdown packages.
4	find_all_groups	backlog_tools.py	BacklogGrouper	Find all possible groupings (equipment, area, shutdown) at once.
5	stratify_backlog	backlog_tools.py	BacklogGrouper	Stratify backlog by priority, reason, and equipment criticality.
6	generate_sap_upload	sap_tools.py	SAPExportEngine	Generate SAP PM upload template from work packages.
7	validate_sap_cross_references	sap_tools.py	SAPExportEngine	Validate cross-references in SAP upload data.
8	validate_sap_field_lengths	sap_tools.py	SAPExportEngine	Validate SAP field length constraints.
9	generate_work_instruction	work_instruction_tools.py	WorkInstructionGenerator	Generate a structured work instruction.
10	validate_work_instruction	work_instruction_tools.py	WorkInstructionGenerator	Validate a work instruction for completeness.
11	validate_wp_name	validation_tools.py	NamingValidator	Validate work package name against naming rules (WP-05 to WP-07).
12	validate_task_name	validation_tools.py	NamingValidator	Validate task name against naming rules (T-05 to T-19).
13	validate_work_packages	validation_tools.py	QualityValidator	Validate work package data quality.
14	create_capa	capa_tools.py	CAPAEEngine	Create a new CAPA record with PDCA lifecycle.
15	advance_capa_phase	capa_tools.py	CAPAEEngine	Advance CAPA through PDCA phases (PLAN->DO->CHECK->ACT).
16	update_capa_status	capa_tools.py	CAPAEEngine	Update CAPA status (OPEN/IN_PROGRESS/COMPLETED/OVERDUE).
17	add_capa_action	capa_tools.py	CAPAEEngine	Add an action item to a CAPA record.
18	set_capa_root_cause	capa_tools.py	CAPAEEngine	Set the root cause for a CAPA.
19	check_capa_overdue	capa_tools.py	CAPAEEngine	Check if a CAPA is overdue.
20	get_capa_summary	capa_tools.py	CAPAEEngine	Get summary of a CAPA record.
21	calculate_planning_kpis	planning_kpi_tools.py	PlanningKPIEngine	Calculate 11 GFSN planning KPIs.
22	get_planning_kpi_targets	planning_kpi_tools.py	PlanningKPIEngine	Get target values for all planning KPIs.
23	create_weekly_program	scheduling_tools.py	SchedulingEngine	Create a DRAFT weekly program from work packages.
24	level_program_resources	scheduling_tools.py	SchedulingEngine	Run resource leveling on a weekly program.
25	detect_scheduling_conflicts	scheduling_tools.py	SchedulingEngine	Detect resource conflicts (area interference, specialist overallocation).
26	validate_wp_elements	scheduling_tools.py	SchedulingEngine	Validate 7 mandatory work package elements per REF-14 section 5.5.
27	finalize_weekly_program	scheduling_tools.py	SchedulingEngine	Transition weekly program from DRAFT to FINAL.
28	generate_gantt	scheduling_tools.py	GanttGenerator	Generate Gantt chart data from a weekly program.
29	analyze_spare_parts	reliability_tools.py	SparePartsEngine	Classify spare parts using VED/FSN/ABC methodology.
30	calculate_stock_levels	reliability_tools.py	SparePartsEngine	Calculate reorder point, EOQ, and safety stock levels.
31	create_shutdown	reliability_tools.py	ShutdownEngine	Create a new shutdown record.

#	Tool Name	Wrapper File	Engine(s) Wrapped	Description
32	update_shutdown_progress	reliability_tools.py	ShutdownEngine	Update shutdown execution progress.
33	complete_shutdown	reliability_tools.py	ShutdownEngine	Complete a shutdown and calculate final metrics.
34	calculate_shutdown_metrics	reliability_tools.py	ShutdownEngine	Calculate SPI, CPI, and variance metrics for a shutdown.
35	create_moc	reliability_tools.py	MoCEngine	Create a new Management of Change record.
36	advance_moc	reliability_tools.py	MoCEngine	Advance MoC through its approval workflow.
37	assess_moc_risk	reliability_tools.py	MoCEngine	Assess risk for a Management of Change.
38	calculate_ocr	reliability_tools.py	OCREngine	Calculate Optimum Cost-Risk maintenance interval.
39	analyze_jackknife	reliability_tools.py	JackKnifeEngine	Run Jack-Knife analysis (MTBF vs MTTR 4-zone classification).
40	analyze_pareto	reliability_tools.py	ParetoEngine	Run Pareto 80/20 bad actor analysis.
41	calculate_lcc	reliability_tools.py	LCCEngine	Calculate Life Cycle Cost (NPV per ISO 15663-1).
42	compare_lcc_alternatives	reliability_tools.py	LCCEngine	Compare LCC for multiple alternatives with breakeven analysis.
43	assess_rbi	reliability_tools.py	RBIEngine	Assess Risk-Based Inspection requirements.
44	prioritize_inspections	reliability_tools.py	RBIEngine	Prioritize inspections by risk ranking.
45	validate_import_data	reporting_tools.py	DataImportEngine	Validate data for import (hierarchy, failures, plans).
46	export_equipment_data	reporting_tools.py	DataExportEngine	Export equipment data in structured format.
47	export_kpi_data	reporting_tools.py	DataExportEngine	Export KPI data in structured format.
48	export_report_data	reporting_tools.py	DataExportEngine	Export report data in structured format.
49	export_schedule_data	reporting_tools.py	DataExportEngine	Export schedule data in structured format.
50	check_backlog_aging	notification_tools.py	NotificationEngine	Check for aged backlog items exceeding thresholds.
51	assemble_work_package	wp_assembly_tools.py	WorkPackageAssemblyEngine	Assemble a work package with 7 mandatory elements.
52	check_wp_element_readiness	wp_assembly_tools.py	WorkPackageAssemblyEngine	Check element readiness issues for an assembled work package.
53	generate_wp_compliance_report	wp_assembly_tools.py	WorkPackageAssemblyEngine	Generate compliance report across multiple assembled work packages.
54	build_execution_sequence	execution_task_tools.py	ExecutionTaskEngine	Build ordered execution sequence with dependencies and safety checklists.
55	level_resources_enhanced	scheduling_tools.py	SchedulingEngine	Trade-specific resource leveling with multi-day splitting.

B.4 -- Spare Parts Agent Tools (3 tools)

#	Tool Name	Wrapper File	Engine(s) Wrapped	Description
1	suggest_materials	material_tools.py	MaterialMapper	Suggest spare parts/materials for a component and failure mode.
2	validate_task_materials	material_tools.py	MaterialMapper	Validate that materials assigned to a task are appropriate.
3	resolve_equipment	equipment_tools.py	EquipmentResolver	Resolve equipment TAG from voice/text input using fuzzy matching.

B.5 -- Tools Not Assigned to Any Agent (registered but not in AGENT_TOOL_MAP)

#	Tool Name	Wrapper File	Engine(s) Wrapped	Description
1	get_all_entity_states	state_machine_tools.py	StateMachine	Get all possible states for an entity type.
2	validate_hierarchy	validation_tools.py	QualityValidator	Validate hierarchy data structure.
3	validate_functions	validation_tools.py	QualityValidator	Validate function descriptions.
4	validate_criticality_data	validation_tools.py	QualityValidator	Validate criticality assessment data.

Section C -- Validators

All validators are located under `tools/validators/`.

#	Validator Name	File Path	Rules Count	Key Methods	Description
1	QualityValidator	tools/validators/quality_validator.py	40+	<code>validate_hierarchy()</code> , <code>validate_functions()</code> , <code>validate_criticality()</code> , <code>validate_failure_modes()</code> , <code>validate_tasks()</code> , <code>validate_work_packages()</code> , <code>validate_cross_entity()</code> , <code>validate_frequency_alignment()</code> , <code>validate_suppressive_wp()</code> , <code>validate_sequential_wp()</code> , <code>validate_mi_replacement_tasks()</code> , <code>validate_criticality_fm_alignment()</code> , <code>validate_wp_frequency_alignment()</code> , <code>run_full_validation()</code>	Master data quality validator implementing rules from REF-04. Validates all entity types (hierarchy nodes, functions, criticality matrices, failure modes, maintenance tasks, work packages) and cross-entity referential integrity. Returns ValidationResult with errors, warnings, and info-level issues per entity.
2	ConfidenceValidator	tools/validators/confidence_validator.py	20 (4 levels x 5 entity types)	<code>evaluate()</code> , <code>batch_evaluate()</code>	Evaluates AI confidence scores against thresholds for 4 review levels: NO_REVIEW ($\geq 0.85-0.90$), SPOT_CHECK ($\geq 0.70-0.75$), FULL_REVIEW ($\geq 0.50-0.55$), REJECT ($< 0.50-0.55$). Supports 5 entity types: hierarchy, function, criticality, failure_mode, task. Returns ConfidenceResult with review level and threshold details.
3	NamingValidator	tools/validators/naming_validator.py	12+	<code>validate_wp_name()</code> , <code>validate_task_name()</code> , <code>validate_fm_what()</code>	Validates naming conventions per GFSN standards. Work package names (rules WP-05 to WP-07): mandatory prefix format, max length, allowed characters. Task names (rules T-05 to T-19): verb-first structure, required frequency suffix, max length. Failure mode "what" (rules FM-01, FM-02): must match maintainable item naming patterns. Returns list of NamingIssue (rule_id, severity, message).

Section D -- Processors

All processors are located under `tools/processors/` .

#	Processor Name	File Path	Key Methods	Module	Description
1	PII Redactor	<code>tools/processors/pii_redactor.py</code>	<code>redact()</code>	Data Preprocessing	Regex-based PII redaction before AI processing. Supports English, and Arabic (M./Mme/Dr/In- addresses, phone numbers, email IDs (EMP/MAT/INT), detects "reported" attributions. Returns list of redacted items. No LLM required.
2	FieldCaptureProcessor	<code>tools/processors/field_capture_processor.py</code>	<pre> process(), _extract_text(), _resolve_equipment(), _detect_failure_mode(), _detect_component(), _detect_safety_flags(), _determine_wo_type(), _suggest_spare_parts(), _analyse_images(), _build_structured_description(), _determine_specialties(), _estimate_duration() </pre>	Field Capture Pipeline	Converts raw field data (voice/text/images) into structured work order (DRAFT status). Performs text extraction, resolution (fuzzy mode detection against 72 VALL using MECHANICAL CAUSE KEYWORDS), component type flag detection - determination - spare parts suggestion analysis -> structured (EN+FR). Determination required.
3	PlannerEngine	<code>tools/processors/planner_engine.py</code>	<pre> recommend() (plus 7 private helper functions: _analyse_workforce(), _check_materials(), _find_shutdown_window(), _estimate_production_impact(), _find_groupable(), _suggest_schedule(), _assess_risk(), _determine_action()) </pre>	Planning Recommendations	Generates AI recommended work order validated work order, workforce availability, inventory, shutdown production impact, groupability, and Returns Plan with suggested (APPROVE/MODIFY) scheduling suggestions, group analysis, risk assessment (CRITICAL/HIGH-confidence score). LLM required.
4	BacklogOptimizer	<code>tools/processors/backlog_optimizer.py</code>	<pre> optimize() (plus 7 private helper functions: _to_backlog_entries(), _stratify(), _build_work_packages(), _schedule_shutdown_items(), _generate_schedule(), _generate_alerts(), _empty_backlog()) </pre>	Backlog Optimization	Generates optimized backlog items. Distributes work packages schedulable vs. non-schedulable. Distributes packages shutdown schedules shutdown window alerts (overdue delays, emergency escalations). Returns OptimizedBacklog stratification, work order proposal, and a LLM required.
5	GanttGenerator	<code>tools/processors/gantt_generator.py</code>	<pre> generate_gantt_data(), export_gantt_excel() </pre>	Visualization	Converts Week packages into structured data and export two sheets: (1) Headers/data are color-coded time (MECHANICAL= ELECTRICAL=yellow INSTRUMENTA

#	Processor Name	File Path	Key Methods	Module	Description
					WELDING=orac Uses openpyxl f Deterministic, n

Section E -- Generators

All generators are located under `tools/generators/`.

#	Generator Name	File Path	Key Methods	Description
1	SyntheticDataGenerator	<code>tools/generators/synthetic_data.py</code>	<code>generate_plant_hierarchy()</code> , <code>generate_failure_modes()</code> , <code>generate_work_order_history()</code> , <code>get_statistics()</code>	Generates phosphate-realistic synthetic maintenance data for OCP mining operations. Creates complete 6-level plant hierarchies (PLANT -> AREA -> SYSTEM -> EQUIPMENT -> SUB_ASSEMBLY -> MAINTAINABLE_ITEM) with domain-accurate data: 8 phosphate process areas (Grinding, Flotation, Sedimentation, Filtration, Drying, Conveying, Storage, Pumping), realistic equipment types with manufacturers (FLSmidth, Metso Outotec, Weir Minerals, SKF, ABB, Siemens, etc.), sub-assemblies, maintainable items, failure modes mapped to failure patterns and maintenance strategies, and synthetic work order history. Seeded RNG for reproducibility.

Appendix: File Index

Engines ([tools/engines/](#))

File	Engine Class
backlog_grouper.py	BacklogGrouper
capa_engine.py	CAPAEEngine
criticality_engine.py	CriticalityEngine
cross_module_engine.py	CrossModuleEngine
data_export_engine.py	DataExportEngine
data_import_engine.py	DataImportEngine
de_kpi_engine.py	DEKPIEngine
equipment_resolver.py	EquipmentResolver
execution_task_engine.py	ExecutionTaskEngine
fmeca_engine.py	FMECAEngine
health_score_engine.py	HealthScoreEngine
jackknife_engine.py	JackKnifeEngine
kpi_engine.py	KPIEngine
lcc_engine.py	LCCEngine
management_review_engine.py	ManagementReviewEngine
material_mapper.py	MaterialMapper
moc_engine.py	MoCEngine
notification_engine.py	NotificationEngine
ocr_engine.py	OCREngine (Optimum Cost-Risk)
pareto_engine.py	ParetoEngine
planning_kpi_engine.py	PlanningKPIEngine
priority_engine.py	PriorityEngine
rbi_engine.py	RBIEngine
rca_engine.py	RCAEngine
rcm_decision_engine.py	RCMDecisionEngine
reporting_engine.py	ReportingEngine
scheduling_engine.py	SchedulingEngine
shutdown_engine.py	ShutdownEngine
spare_parts_engine.py	SparePartsEngine
state_machine.py	StateMachine
variance_detector.py	VarianceDetector
weibull_engine.py	WeibullEngine
work_instruction_generator.py	WorkInstructionGenerator
work_package_assembly_engine.py	WorkPackageAssemblyEngine

Tool Wrappers ([agents/tool_wrappers/](#))

File	Tools Registered
backlog_tools.py	5 tools
capa_tools.py	7 tools
criticality_tools.py	4 tools
equipment_tools.py	1 tool
execution_task_tools.py	2 tools
fm_lookup_tools.py	4 tools
fmeca_tools.py	3 tools
health_tools.py	2 tools
kpi_tools.py	5 tools
management_review_tools.py	1 tool
material_tools.py	2 tools
notification_tools.py	6 tools
planning_kpi_tools.py	2 tools
priority_tools.py	2 tools
rca_tools.py	11 tools
rcm_tools.py	2 tools
reliability_tools.py	16 tools
reporting_tools.py	9 tools
scheduling_tools.py	8 tools
sap_tools.py	3 tools
state_machine_tools.py	3 tools
validation_tools.py	13 tools
variance_tools.py	3 tools
weibull_tools.py	3 tools
work_instruction_tools.py	2 tools
wp_assembly_tools.py	3 tools

Validators ([tools/validators/](#))

File	Validator Class
quality_validator.py	QualityValidator
confidence_validator.py	ConfidenceValidator
naming_validator.py	NamingValidator

Processors ([tools/processors/](#))

File	Processor Class
pii_redactor.py	(module-level redact() function)
field_capture_processor.py	FieldCaptureProcessor
planner_engine.py	PlannerEngine
backlog_optimizer.py	BacklogOptimizer
gantt_generator.py	GanttGenerator

Generators ([tools/generators/](#))

File	Generator Class
synthetic_data.py	SyntheticDataGenerator